UNIVERSIDAD PRIVADA DEL NORTE

ESTRUCTURAS DE DATOS

TEMA: **Asociación de Operaciones con Estructuras Dinámicas en una Bodega de Barrio,Los olivos-2026**

DOCENTE: CINTHIA JEANETTE CALDERON AQUIÑO

INTEGRANTES:

- **LOZA CHANTA CHRISTIAN** N00479356@UPN.PE
- **JEREMY ADRIAN ENRICO MIRAVAL N00483037@UPN.PE**
- **JEFFRY JAIR LIONEL GUERRERO GUARDAMINO N00465756**@UPN.PE
- **YOEL ANDERSON GIRALDO GIRALDO N00484909**@UPN.PE



Los Olivos - LIMA, PERÚ 2026

ÍNDICE

PORTADA.....1

DESARROLLO.....3

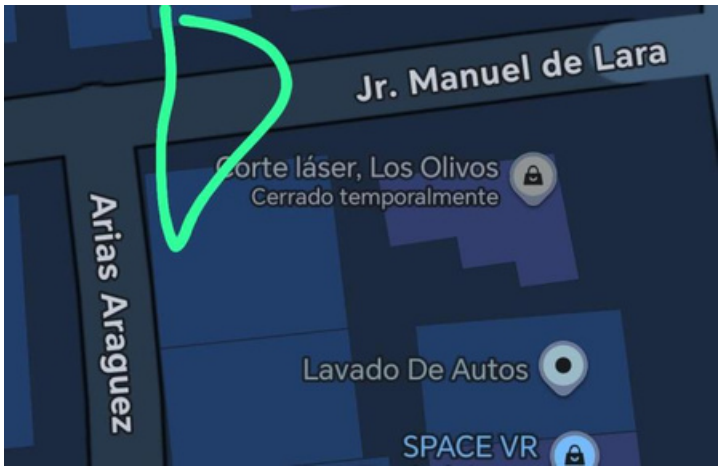
1. OBSERVACION FOTOGRAFICA RESGITRADA.....3
2. PROBLEMA TRANSACCIONAL Y DIAGRAMA.....6
3. SELECCIÓN Y JUSTIFICACIÓN DE ESTRUCTURAS 10
4. DISEÑO LÓGICO 20
5. IMPLEMENTACIÓN MODULAR 25
6. CASOS DE PRUEBA 28

DESARROLLO

1. Observacion fotografica resgitrada :

El lugar de estudio fue en una bodega de barrio ubicada en el cruce del Jr. Manuel de Lara con la calle Arias Aráguez. El negocio es atendido por una sola persona, sin empleados. El horario de atención reportado es de 11:00 a. m. a 12:30 a. m. La recepción de mercadería como gaseosas, papas, galletas y mas se almacena en un cuarto habilitado en la vivienda de la propietaria. Durante la visita se registraron las operaciones de compra realizadas por los clientes y el medio de pago utilizado en cada una.

Bodega ubicada en el cruce del Jr. Manuel de Lara



Bodega en uso



[1.1] Registro cronológico de eventos

Se creo un cuadro cronologico de hora donde se registraba los diferentes clientes, mostrando tamien la categoria de su llega y pago correspondiente a travez de efecto, Yape y Plin

Hora	Evento observado	Categoria
5:40 p.m	Cola de 3 personas	Llegada
5:40 p.m	1ª persona de la cola pagó con Yape	Pago
5:40 p.m	2ª persona de la cola pagó en efectivo	Pago
5:40 p.m	3ª persona de la cola [medio de pago no registrado]	Pago
5:45 p.m	1 persona llegó y pagó con Yape	Llegada / Pago
5:47 p.m	1 persona llegó y pagó en efectivo	Llegada / Pago
5:48 p.m	1 persona llegó y pagó con Plin	Llegada / Pago
5:48 p.m	1 persona llegó y pagó con Yape	Llegada / Pago
5:59 p.m	Cola de 4 personas	Llegada
5:59 p.m	1ª persona de la cola pagó con Yape	Pago
5:59 p.m	2ª persona de la cola pagó con Yape	Pago
5:59 p.m	3ª persona de la cola pagó en efectivo	Pago
5:59 p.m	4ª persona de la cola pagó con Yape	Pago

[1.2] Registro analisado

LLEGADA

Observacion	Significado	Acciones
Durante la visita llegaron varios clientes y en algunos momentos se formó una cola.	Esto quiere decir que hay bastante movimiento de clientes.	Anotar las horas en que llegan más clientes para saber cuándo hay mayor demanda

ESPERAS

Observacion	Significado	Acciones
Algunos clientes tuvieron que esperar porque había otras personas comprando.	Puede hacer que la atención sea un poco más lenta.	Observar cuánto tiempo esperan los clientes y buscar una forma de atenderlos más rápido.

RESERVAS

Observacion	Significado	Acciones
No vimos ningún pago reversado durante nuestra visita.	No tuvimos un caso para poder analizarlo.	Registrar cuando ocurra una reversa para poder saber como se soluciona.

CONSULTAS

Observacion	Significado	Acciones
Observamos un consulta.	Esto significa que los clientes que atendimos ya sabían qué producto querían comprar.	Registrar las consultas que hagan los clientes en una próxima visita.

2. Problema Transaccional y diagrama :

Durante 4 de septiembre de 2026, de 5:40 p.m. a 5:59 p.m., en una bodega ubicada en el cruce del Jr. Manuel de Lara con la calle Arias Aráguez. Asistieron integrantes del equipo, quienes anotaron 11 pagos: 7 con billetera móvil (6 con Yape y 1 con Plin) y 4 en efectivo donde se indentifico un problema transaccional.

Como Yape y Plin dependen del internet, si la señal falla o va lenta el cliente tiene que esperar o volver a intentar, quienes están detrás también esperan y la venta puede quedar pendiente o mal anotada.Durante la observacion no se presento por el momento cortes de internet pero si advertimos que los pagos con billeteras móviles pueden generar esperas o reintentos inesperados



foto qr

[2.1] Análisis de entradas, procesos y salidas

ENTRADAS

Variable	Descripción	Validación
ID Transacción	Código único de cada compra.	Obligatorio y único; lo genera el sistema
Fecha/Hora	Momento de la compra.	Formato válido
Usuario	Persona que atiende y registra la compra.	Usuario que entró con su clave
Cliente	Persona que compra.	Opcional; si se registra, que exista en la lista de clientes
Monto	Cuánto paga el cliente.	Mayor a 0
Medio de pago	Cómo paga: Yape, Plin o efectivo.	Obligatorio; solo uno de los tres
Tipo de operación	Qué hace el cliente: compra, consulta o reversa.	Solo los valores de la lista
Estado de conexión	Si hay internet o no en ese momento.	Con internet (online) o sin internet (offline)

PROCESOS

Comienza cuando la persona que atiende ingresa la hora, el monto, el medio de pago además de el tipo de operación para luego revisar que no falte nada y que el monto sea mayor a 0. Después se aplican las reglas de la bodega: se propone aceptar solo Yape, Plin o efectivo, y dar por pagado un pago con billetera cuando llegue su confirmación. El sistema le asigna a cada compra un código único y la guarda primero en el mismo equipo que usa la bodega. Con la compra guardada, el sistema comprueba si hay internet en ese momento. Si lo hay, la envía al sistema principal, donde queda almacenada, y este avisa que la recibió. Recién entonces la compra pasa a estar confirmada.

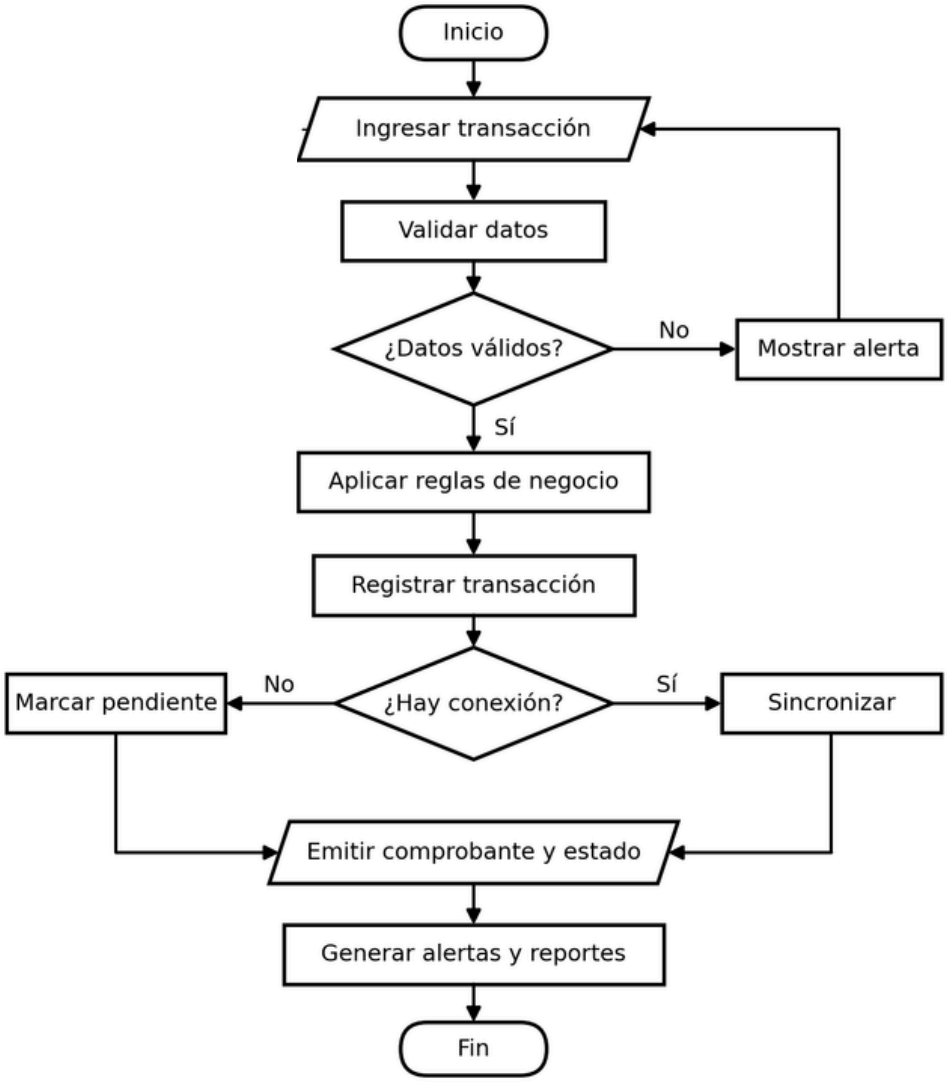
SALIDA

Al terminar, la compra queda registrada con su código y con un comprobante que sirve de constancia de la venta. Cada compra muestra si está confirmada o pendiente. El sistema guarda además una bitácora con la hora y lo que pasó, emite alertas cuando una compra queda pendiente, cuando no hay internet o cuando falta algún dato y con esa información se arman reportes sobre las horas con más clientes, los pagos por cada medio, las esperas, las reversas y las consultas.

MANEJO DE CONECTIVIDAD INTERMITENTE

Con internet, la compra se registra y se envía al momento. Si la señal se corta y vuelve, la compra queda guardada en el equipo de la bodega y el sistema intenta enviarla otra vez por su cuenta sin que quien atiende tenga que hacer nada. Si no hay conexión, la compra espera en una cola de pendientes dentro del equipo y se envía apenas vuelve el internet, igual que la cola de personas que los integrantes vieron en la bodega: la primera compra en quedar pendiente es la primera en enviarse.

[2.2] Diagrama Entradas-Procesos-Salidas



De esta manera se llegaría a optimizar las ventas en situaciones como cuando el personal anota citas y diagnósticos y si falla la red, la información se guarda un tiempo en el equipo y luego se copia al servidor centra. Se registraría las entradas y salidas desde un dispositivo y la información se enviaría cuando se recupera la señal y así la bodega guardaroa su mercadería

3. Selección y justificación de estructuras

Apartir de las operaciones observadas en la bodega (sección 1) y del análisis de entradas, procesos y salidas (sección 2), se asocia cada operación con una estructura dinámica candidata y se justifica por qué supera a una estructura estática en uso de memoria y eficiencia. En un arreglo estático la capacidad se fija de antemano y insertar o eliminar en posiciones intermedias obliga a desplazar elementos; en una estructura dinámica la memoria crece y se libera nodo a nodo según la necesidad real del negocio.

[3.1] Matriz operación-estructura

Operación	Estructura elegida	Motivo de la elección	Ventaja frente a una estructura estática
Operación 1: Gestionar la fila de clientes	**Cola con lista enlazada simple (cabeza y final)**	Atención por orden de llegada; insertar al final y eliminar el primero en O(1), con un solo puntero por nodo.	Memoria proporcional a los clientes en espera; sin desborde en hora pico ni desperdicio en hora valle.
Operación 2: Registrar mercadería recibida	**Lista enlazada simple**	Inserción al inicio en O(1) y recorrido secuencial para consultar existencias; es la más ligera en memoria.	Sin capacidad fija ni copias al crecer; los ingresos de mercadería son impredecibles.
Operación 3: Registrar y anular pagos	**Lista enlazada doble (cabeza y final)**	Agregar y quitar el último pago en O(1), porque cada nodo conoce a su anterior.	Memoria solo por los pagos reales; quitar un pago no deja huecos ni exige desplazar elementos.
Operación 4: Ronda de revisión de existencias	**Lista enlazada circular**	El último nodo apunta al primero: la ronda de revisión se repite sin reiniciar el recorrido.	Sin límite de capacidad ni control de índices con módulo; agregar o quitar categorías solo reasigna enlaces.
Operación 5: Revisar y corregir el historial de pagos	**Lista enlazada doble**	Navegación en ambos sentidos e inserción o eliminación de un nodo conocido en O(1).	Insertar un registro intermedio reasigna dos enlaces, sin desplazar el arreglo (O(n)).

[3.2] Estructuras candidatas evaluadas

Para cada operación se compararon las estructuras dinámicas candidatas (lista enlazada, pila, cola y árbol de búsqueda binaria). La celda sombreada indica la estructura elegida.

Operación	Lista enlazada	Pila	Cola	Árbol de búsqueda binaria
Op. 1: Fila de clientes	Sirve, pero el orden FIFO lo da la cola.	No: atendería primero al último que llegó (LIFO).	ELEGIDA: orden de llegada; se implementa sobre lista simple.	No: no hay clave que ordenar; costo innecesario.
Op. 2: Mercadería recibida	ELEGIDA (simple): inserta al inicio en O(1) y se recorre para consultar.	No: solo da acceso al último ingreso.	No: obligaría a consultar en orden de ingreso.	Posible si el catálogo crece mucho, pero exige balanceo y más punteros.

Operación	Lista enlazada	Pila	Cola	Árbol de búsqueda binaria
Op. 3: Registrar y anular pagos	ELEGIDA (doble): quita el último en O(1).	Anula el último (LIFO), pero no permite recorrer ni corregir el historial.	No: anula el más reciente, no el primero.	No: los pagos son cronológicos, no se buscan por clave.
Op. 4: Ronda de revisión	ELEGIDA (circular): la ronda se repite sin reiniciar.	No: consumiría los elementos al recorrerlos.	Posible (sacar y reinsertar), pero mueve cada nodo en cada vuelta.	No: la ronda es cíclica, no hay orden de comparación.
Op. 5: Corregir historial	ELEGIDA (doble): navega en ambos sentidos e inserta en O(1).	No: solo accede al tope, no a un registro intermedio.	No: solo accede a los extremos.	Permitiría buscar por ID, pero exige balanceo y no refleja la secuencia cronológica.

[3.3] Justificación técnica y código por operación

Operación N.° 1: Gestionar la fila de clientes

Operación identificada: Registrar la llegada de cada cliente y atenderlo en orden de llegada.

Estructura dinámica: Cola implementada con lista enlazada simple (punteros a la cabeza y al final). **Justificación:** La fila funciona como cola (primero en llegar, primero en ser atendido) con un solo atendedor. La lista inserta al final y elimina el primero sin mover a los demás clientes, y como solo se avanza en un sentido, un puntero por nodo basta. Cada nodo guarda la hora de llegada y un identificador anónimo, sin datos personales. La misma lógica de orden se aplica a la cola de compras pendientes descrita en el manejo de conectividad intermitente (sección 2.1).

Ventaja frente a una estructura estática: Un arreglo estático tiene tamaño fijo: si es grande, desperdicia memoria en horas valle (0 a 2 clientes); si es pequeño, se desborda en hora pico. Además, quitar al primer cliente obliga a desplazar a todos los demás. La lista usa un nodo por cliente real y libera la memoria al atenderlo.

Código C#:

```
9       public class NodoCliente
10      {
11          //operación 1
            1 referencia
12          public int Turno;
            1 referencia
13          public DateTime Hora;
            6 referencias
14          public NodoCliente Siguiente;
15
            1 referencia
16          public NodoCliente(int turno, DateTime hora)
17          {
18              this.Turno = turno;
19              this.Hora = hora;
20              this.Siguiente = null;
21          }
22      }
23  }
24
```

Clase NodoCliente (Operación 1)

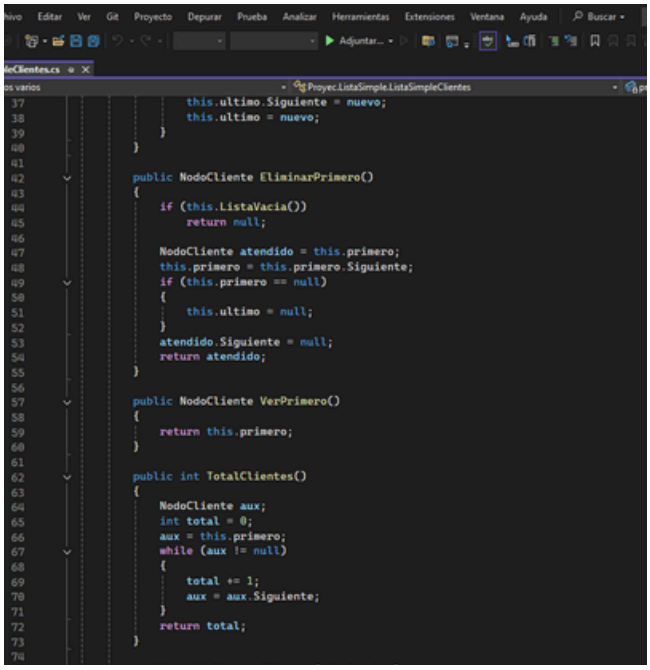
```
namespace Proyec.ListaSimple
{
    public class ListaSimpleClientes
    {
        //operación 1
        private NodoCliente primero;
        private NodoCliente ultimo;

        public ListaSimpleClientes()
        {
            this.primero = null;
            this.ultimo = null;
        }

        public bool ListaVacia()
        {
            return this.primero == null;
        }

        public void InsertarDetras(int turno, DateTime hora)
        {
            NodoCliente nuevo = new NodoCliente(turno, hora);
            if (this.ListaVacia())
            {
                this.primero = nuevo;
                this.ultimo = nuevo;
            }
            else
            {
                this.ultimo.Siguiente = nuevo;
                this.ultimo = nuevo;
            }
        }
```

Clase ListaSimpleClientes: constructor, ListaVacia e InsertarDetras



```
            this.ultimo.Siguiente = nuevo;
            this.ultimo = nuevo;
        }
    }

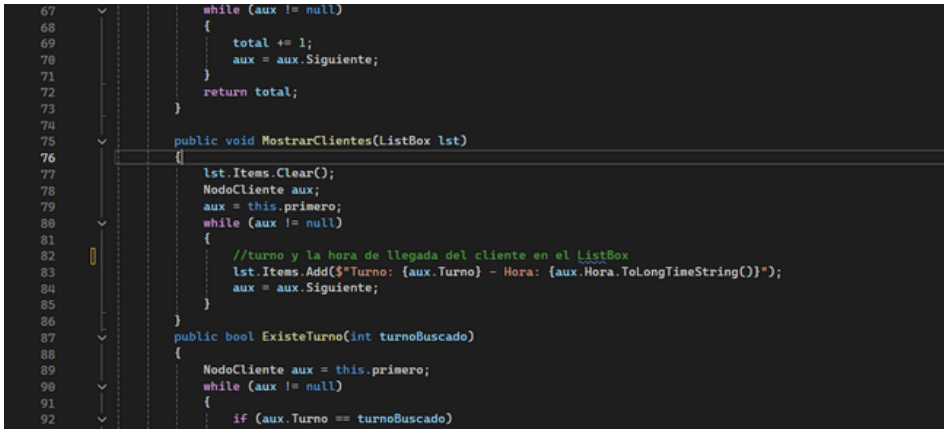
    public NodoCliente EliminarPrimero()
    {
        if (this.ListaVacia())
            return null;

        NodoCliente atendido = this.primero;
        this.primero = this.primero.Siguiente;
        if (this.primero == null)
        {
            this.ultimo = null;
        }
        atendido.Siguiente = null;
        return atendido;
    }

    public NodoCliente VerPrimero()
    {
        return this.primero;
    }

    public int TotalClientes()
    {
        NodoCliente aux;
        int total = 0;
        aux = this.primero;
        while (aux != null)
        {
            total += 1;
            aux = aux.Siguiente;
        }
        return total;
    }
```

EliminarPrimero, VerPrimero y TotalClientes



```
        while (aux != null)
        {
            total += 1;
            aux = aux.Siguiente;
        }
        return total;
    }

    public void MostrarClientes(ListBox lst)
    {
        lst.Items.Clear();
        NodoCliente aux;
        aux = this.primero;
        while (aux != null)
        {
            //turno y la hora de llegada del cliente en el ListBox
            lst.Items.Add($"Turno: {aux.Turno} - Hora: {aux.Hora.ToLongTimeString()}");
            aux = aux.Siguiente;
        }
    }
    public bool ExisteTurno(int turnoBuscado)
    {
        NodoCliente aux = this.primero;
        while (aux != null)
        {
            if (aux.Turno == turnoBuscado)
```

MostrarClientes en el ListBox

Operación N.° 2: Registrar la mercadería recibida

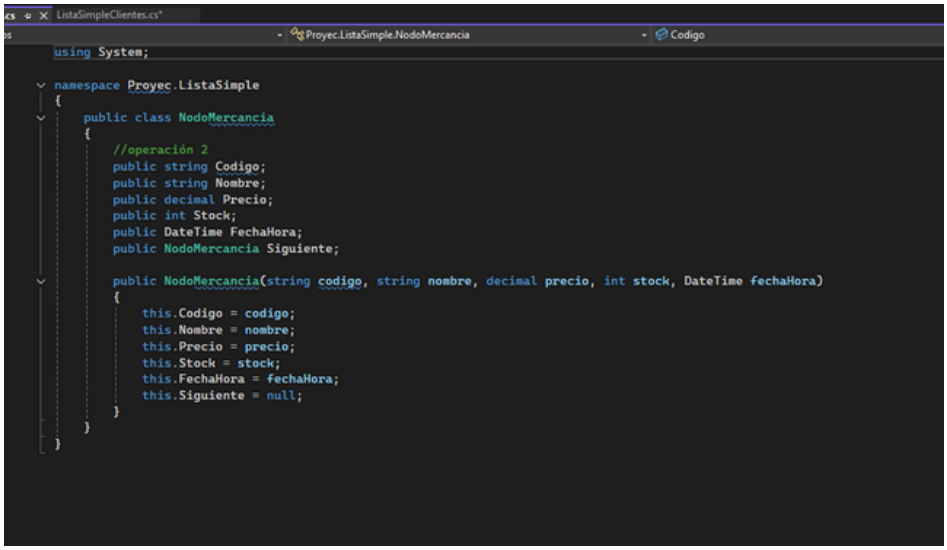
Operación identificada: Registrarla mercaderíarecibida del proveedor (por ejemplo, gaseosas) que se guarda en el cuarto de almacén.

Estructura dinámica: Lista enlazada simple.

Justificación: Cada ingreso se agrega como un nodo nuevo, sin importar el orden. En una lista enlazada simple la inserción al inicio es O(1) (solo se reasigna la cabeza) y el recorrido para consultar existencias es secuencial. Usa un solo puntero por nodo, la opción más ligera en memoria para un registro que solo crece.

Ventaja frente a una estructura estática: La cantidad de ingresos y de productos distintos varía según los pedidos y no se conoce de antemano. Un arreglo debe reservar una capacidad máxima (desperdicio) o ampliarse copiando todo su contenido (O(n)); además, insertar al inicio exige desplazar los elementos. La lista crece nodo a nodo, sin reserva ni copias.

Código C#:



Clase NodoMercancia (Operación 2)

Operación N.° 3: Registrar y anular pagos

Operación identificada: Registrar cada pago (hora y medio: Yape, Plin o efectivo) y anular el último pago registrado cuando hay duplicado, error o reintento fallido.

Estructura dinámica: Lista enlazada doble (punteros a la cabeza y al final).

Justificación: Los pagos se agregan al final en orden cronológico y una reversa casi siempre afecta al más reciente. En una lista doblemente enlazada, agregar y quitar el último nodo es O(1), porque el nodo final conoce a su anterior; en una lista simple, quitar el último exigiría recorrer toda la lista (O(n)) para hallar el predecesor. En la visita no hubo reversas, pero las billeteras pueden generarlas.

Ventaja frente a una estructura estática: En un arreglo estático hay que reservar capacidad para el máximo de pagos de la jornada, con desperdicio los días de poco movimiento y pérdida de registros si se llena. La lista doble ocupa memoria solo por los pagos realmente registrados (11 en 19 minutos) y quitar un pago libera su nodo sin dejar huecos. Su costo es un puntero adicional por nodo.

Código C#:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Proyec.ListaDoble
{
    public class NodoDoblePago
    {
        //operación 3
        public int IdTransaccion;
        public DateTime Hora;
        public string MedioPago;
        public NodoDoblePago Siguiente;
        public NodoDoblePago Anterior;

        public NodoDoblePago(int idTransaccion, DateTime hora, string medioPago)
        {
            this.IdTransaccion = idTransaccion;
            this.Hora = hora;
            this.MedioPago = medioPago;
            this.Siguiente = null;
            this.Anterior = null;
        }
    }
}
```

Clase NodoDoblePago (Operación 3)

```
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace Proyec.ListaDoble
{
    public class ListaDoblePagos
    {
        //operación 3
        private NodoDoblePago primero;
        private NodoDoblePago ultimo;
        public ListaDoblePagos()
        {
            this.primero = null;
            this.ultimo = null;
        }
        public bool ListaVacia()
        {
            return this.primero == null;
        }

        public void InsertarDetras(int idTransaccion, DateTime hora, string medioPago)
        {
            NodoDoblePago nuevo = new NodoDoblePago(idTransaccion, hora, medioPago);
            if (this.ListaVacia())
            {
                this.primero = nuevo;
                this.ultimo = nuevo;
            }
            else
            {
                this.ultimo.Siguiente = nuevo;
                nuevo.Anterior = this.ultimo;
                this.ultimo = nuevo;
            }
        }
        public NodoDoblePago EliminarUltimo()
        {
            if (this.ListaVacia())
```

Clase ListaDoblePagos: constructor, ListaVacia e InsertarDetras

```
public NodoDoblePago EliminarUltimo()
{
    if (this.ListaVacia())
        return null;

    NodoDoblePago eliminado = this.ultimo;
    if (this.primero == this.ultimo)
    {
        this.primero = null;
        this.ultimo = null;
    }
    else
    {
        this.ultimo = this.ultimo.Anterior;
        this.ultimo.Siguiente = null;
        eliminado.Anterior = null;
    }
    return eliminado;
}

public void MostrarHaciaAdelante(ListBox lst)
{
    lst.Items.Clear();
    NodoDoblePago aux = this.primero;
    while (aux != null)
    {
        lst.Items.Add($"ID: {aux.IdTransaccion} - Hora: {aux.Hora:HH:mm:ss} - Pago: {aux.MedioPago}");
        aux = aux.Siguiente;
    }
}

public void MostrarHaciaAtras(ListBox lst)
{
    lst.Items.Clear();
    NodoDoblePago aux = this.ultimo;
    while (aux != null)
    {
        lst.Items.Add($"ID: {aux.IdTransaccion} - Hora: {aux.Hora:HH:mm:ss} - Pago: {aux.MedioPago}");
        aux = aux.Anterior;
    }
}
```

EliminarUltimo y MostrarHaciaAdelante MostrarHaciaAtras

Operación N.° 4: Ronda de revisión de existencias

Operación identificada: Realizar rondasrepetidasderevisión de existencias y reposición de los productos del almacén (gaseosas, abarrotes, etc.).

Estructura dinámica: Lista enlazada circular.

Justificación: La bodega la atiende una sola persona, que revisa los productos por categorías y vuelve a empezar al terminar la ronda. En una lista circular el último nodo apunta al primero, de modo que el recorrido continúa sin reiniciar el puntero ni validar el final de la lista. Agregar o quitar una categoría solo reasigna enlaces, y desde cualquier nodo se puede recorrer toda la ronda.

Ventaja frente a una estructura estática: Con un arreglo estático hay que calcular el índice con módulo y controlar un tamaño máximo, y agregar o quitar una categoría obliga a desplazar elementos (O(n)). La lista circular no tiene límite de capacidad, no requiere control de índices y ajusta la memoria a las categorías que realmente existen.

Código C#:

```
public class NodoCategoria
{
    //operación 4
    2 referencias
    public string Categoria;
    9 referencias
    public NodoCategoria Siguiente;
    1 referencia
    public NodoCategoria(string categoria)
    {
        this.Categoria = categoria;
        this.Siguiente = null;
    }
}
```

Clase NodoCategoria (Operación 4)

```
}
public void AgregarCategoria(string categoria)
{
    NodoCategoria nuevo = new NodoCategoria(categoria);
    if (this.ListaVacia())
    {
        this.ultimo = nuevo;
        this.ultimo.Siguiente = this.ultimo;
        this.actual = nuevo;
    }
    else
    {
        nuevo.Siguiente = this.ultimo.Siguiente;
        this.ultimo.Siguiente = nuevo;
        this.ultimo = nuevo;
    }
}
public bool ExisteCategoria(string categoria)
{
    if (this.ListaVacia())
        return false;

    NodoCategoria aux = this.ultimo.Siguiente;
    do
    {
        if (string.Equals(aux.Categoria, categoria, StringComparison.OrdinalIgnoreCase))
        {
            return true;
        }
        aux = aux.Siguiente;
    } while (aux != this.ultimo.Siguiente);

    return false;
}
```

```
public string RevisarSiguiente()
{
    if (this.ListaVacia())
        return null;

    string categoria = this.actual.Categoria;
    this.actual = this.actual.Siguiente;
    return categoria;
}
```

Clase ListaCircularRonda: AgregarCategoria y RevisarSiguiente

```
public void MostrarRondaRevision(ListBox lst, int vueltas)
{
    lst.Items.Clear();
    if (this.ListaVacia() || vueltas <= 0) return;

    NodoCategoria aux = this.actual;
    int contador = 0;
    int totalNodos = 0;

    NodoCategoria temp = this.ultimo.Siguiente;
    do
    {
        totalNodos++;
        temp = temp.Siguiente;
    } while (temp != this.ultimo.Siguiente);

    do
    {
        lst.Items.Add($"Revisando categoría: {aux.Categoria}");
        aux = aux.Siguiente;
        contador++;
    } while (contador < (vueltas * totalNodos));
}
```

MostrarRondaRevision

Operación N.° 5: Revisar y corregir el historial

Operación identificada: Revisar y corregir el historial de pagos del día, avanzando y retrocediendo entre registros (por ejemplo, completar el medio de pago no registrado de la 3.ª persona de las 5:40 p. m.). **Estructura dinámica:** Lista enlazada doble (la misma lista de la Operación 3, que sirve para ambas tareas). **Justificación:** Corregir exige moverse hacia adelante y hacia atrás entre registros e insertar o eliminar uno intermedio. En una lista doblemente enlazada cada nodo conoce a su anterior y a su siguiente: se navega en ambos sentidos y se inserta o elimina un nodo conocido en O(1). En una lista simple habría que recorrer desde el inicio para hallar al predecesor.

Ventaja frente a una estructura estática: En un arreglo estático, agregar un registro omitido en medio obliga a desplazar todos los elementos siguientes (O(n)) y la capacidad es fija. La lista doble inserta o elimina reasignando solo dos enlaces, y la memoria se ajusta al número real de pagos.

Código C#:

```
        public bool ModificarMedioPago(int idTransaccion, string nuevoMedioPago)
        {
            NodoDoblePago aux = this.primero;
            while (aux != null)
            {
                if (aux.IdTransaccion == idTransaccion)
                {
                    aux.MedioPago = nuevoMedioPago;
                    return true;
                }
                aux = aux.Siguiente;
            }
            return false;
        }
    }
}
```

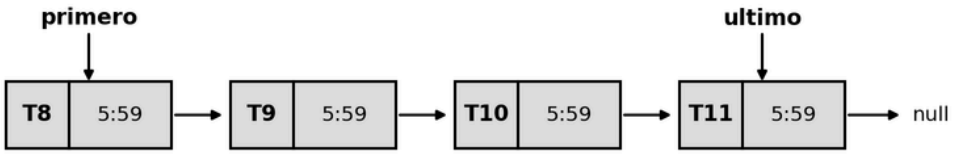
ModificarMedioPago (Operación 5)

4. Diseño lógico :

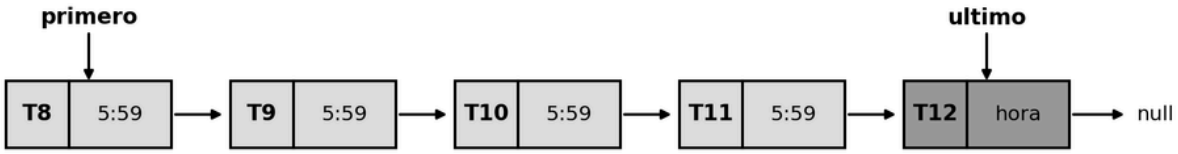
Cada operación se diseña con el diagrama lógico de su estructura (nodos y punteros), el algoritmo de sus operaciones principales y su validación con los datos levantados en campo (4 de septiembre de 2026, de 5:40 p. m. a 5:59 p. m.). Convenciones de los diagramas: [ ] es un nodo, -> es el puntero Siguiente, <- es el puntero Anterior (solo en listas dobles), null marca el fin y primero, ultimo, cabeza y actual son referencias de la lista

[4.1] Operación 1: Cola de clientes

1. Estado de la cola (5:59 p. m.)



2. Encolar: llega el turno 12 (InsertarDetras)

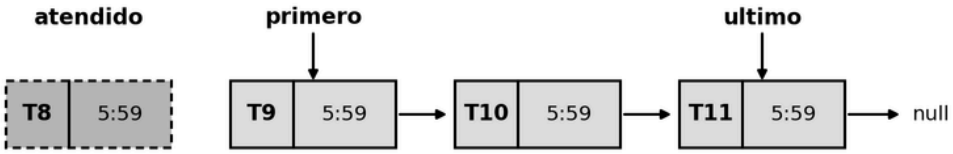


Paso 1: nuevo = [T12|hora|null]

Paso 2: ultimo.Siguiente = nuevo (T11 apunta a T12)

Paso 3: ultimo = nuevo

3. Desencolar: se atiende el turno 8 (EliminarPrimero)



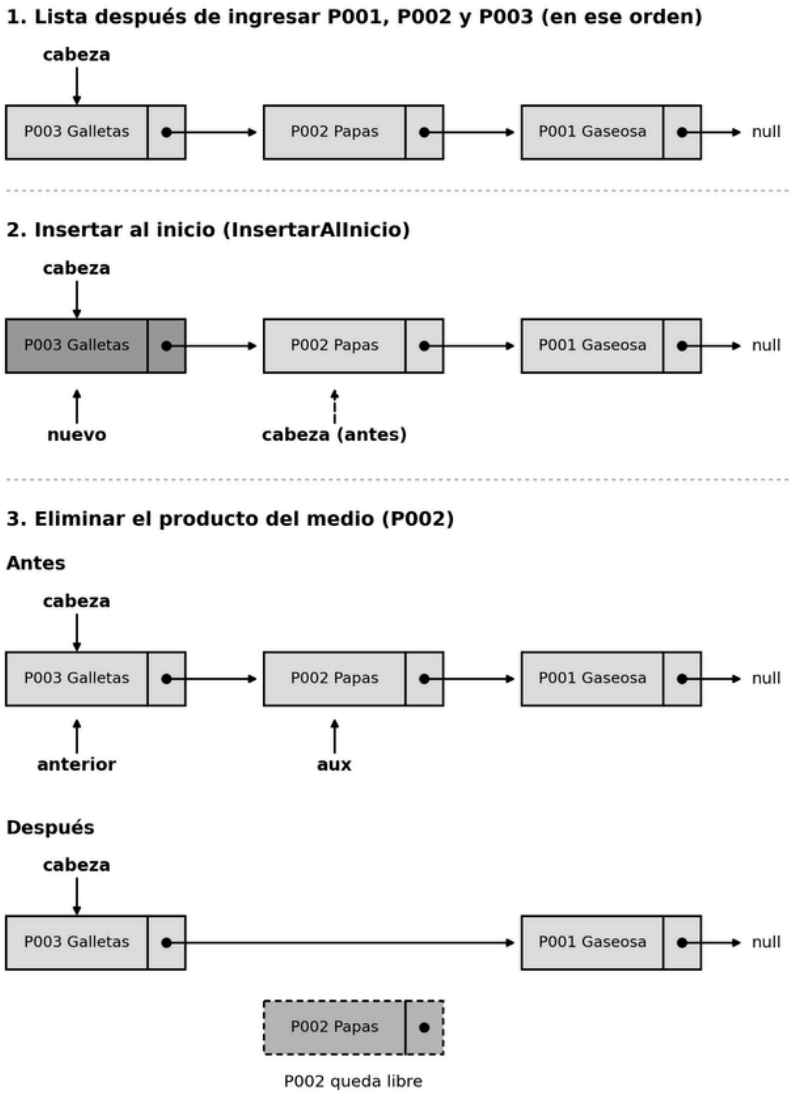
Siguiente = null

[4.2] Operación 2: Mercadería recibida

En la visita se observó mercadería como gaseosas, papas y galletas. Para la prueba se usan esos tres productos con códigos propios; los precios y el stock son valores referenciales

Codigo	Producto	Precio (S/)	Stock
P001	Gaseosa	3.50	24
P002	Papas	1.50	30
P003	Galletas	1.00	40

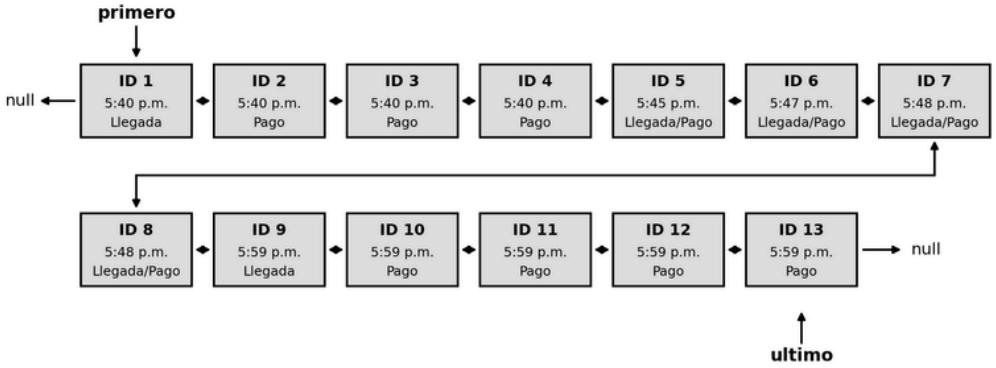
Cada ingreso nuevo queda al inicio, por eso el último registrado P003 se muestra primero



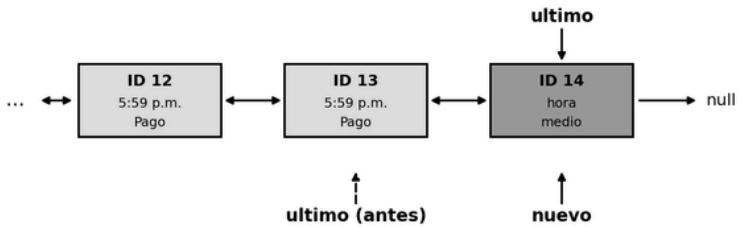
[4.3] Operación 3: Registrar y anular pagos

Los 11 pagos del registro [1.1] se cargan en orden cronológico y el sistema asigna un ID único creciente. El medio de pago de la 3.ª persona de las 5:40 p. m. no se registró en campo; en el prototipo se carga de forma provisional como Efectivo para corregirlo después con la Operación 5. Con ese valor provisional, el total por medio coincide con el de la sección 2: 6 Yape, 1 Plin y 4 Efectivo

1. Historial de pagos de la visita (13 nodos)

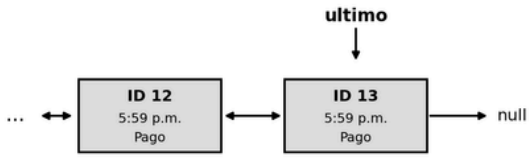


2. Registrar un pago (InsertarDetras): ID 14

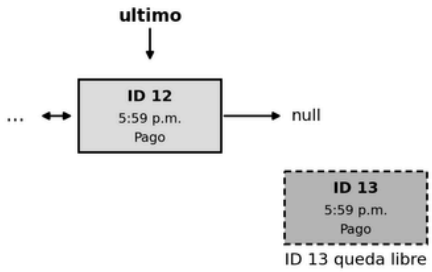


3. Anular el último pago (EliminarUltimo): reversa del ID 13

Antes



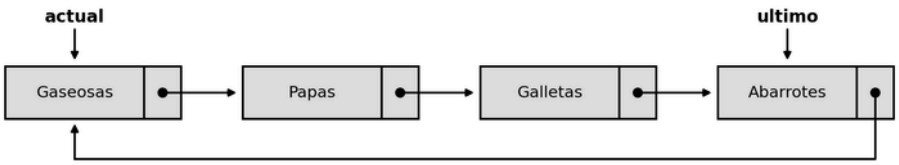
Después



[4.4] Operación 4: Ronda de revisión

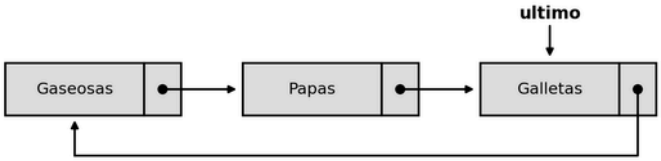
La propietaria repone gaseosas, papas, galletas y abarrotes (sección 1). Esas cuatro categorías forman la ronda; al terminar la última se vuelve a la primera

1. Ronda con 4 categorías (lista circular)

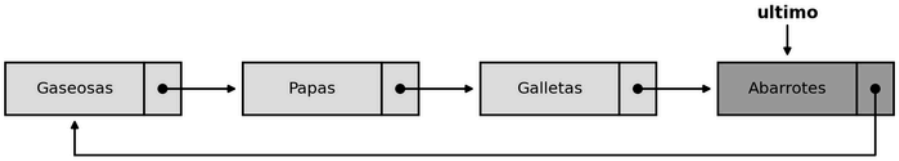


2. Agregar una categoría (AgregarCategoria): se agrega Abarrotes al final

Antes

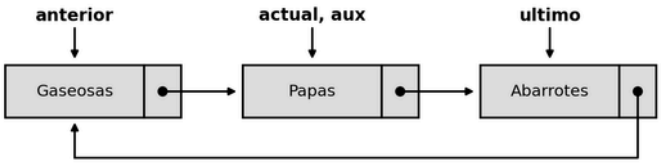


Después

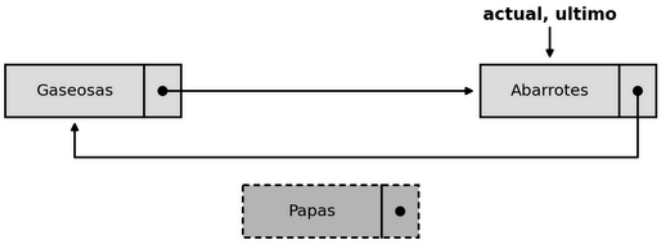


3. Quitar una categoría (EliminarCategoria): se quita Papas

Antes



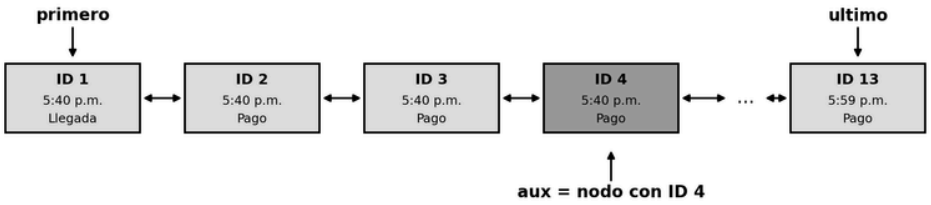
Después

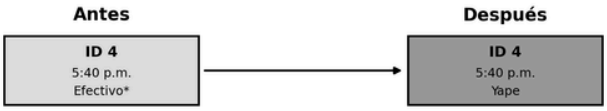


[4.5] Operación 5: Revisar

Los pagos del registro se cargan en orden cronológico y el sistema asigna un ID único creciente. Con ese valor provisional, el total por medio se obtiene por medios de: 6 Yape, 1 Plin y 4 Efectivo

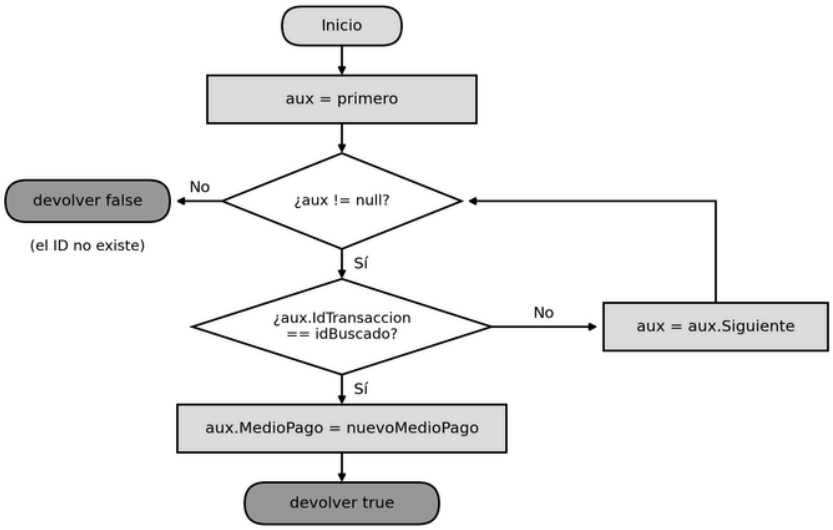
1. Localizar y corregir el ID 4



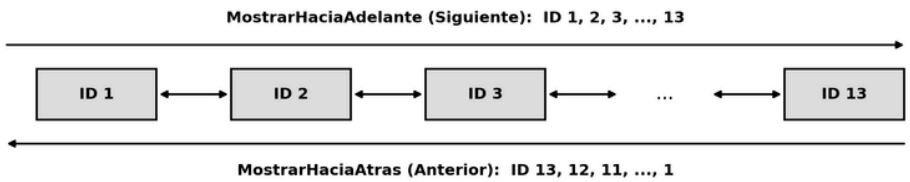


Solo cambia el medio de pago; los enlaces no se tocan.
* Efectivo provisional (no se registró en campo).

2. Algoritmo ModificarMedioPago



3. Verificación en ambos sentidos



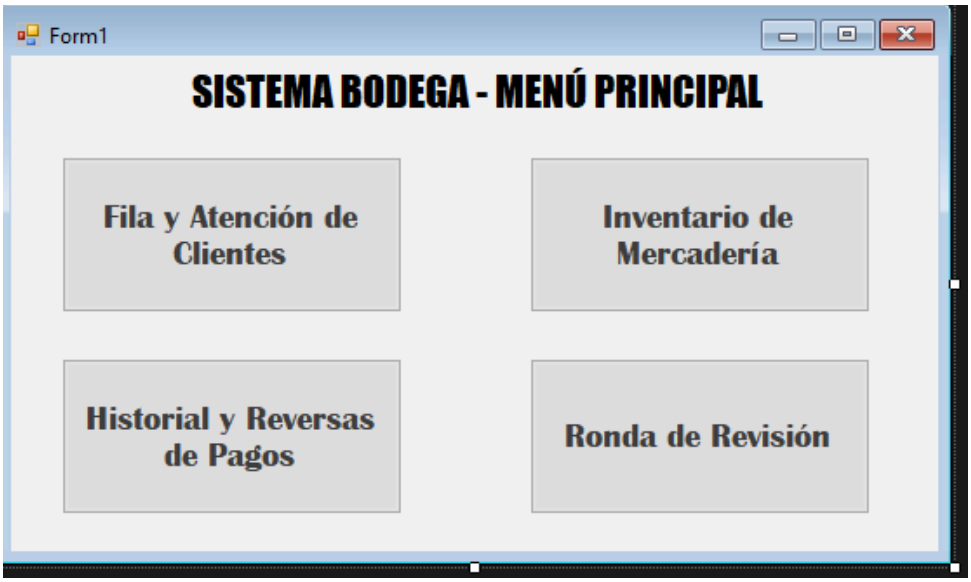
5. Implementación modular :

Se creo un prototipo funcional programado en C#, donde cada dato tiene en su propia clase, separada de los formularios: la clase de la lista y el formulario que solo captura datos, valida y muestra resultados

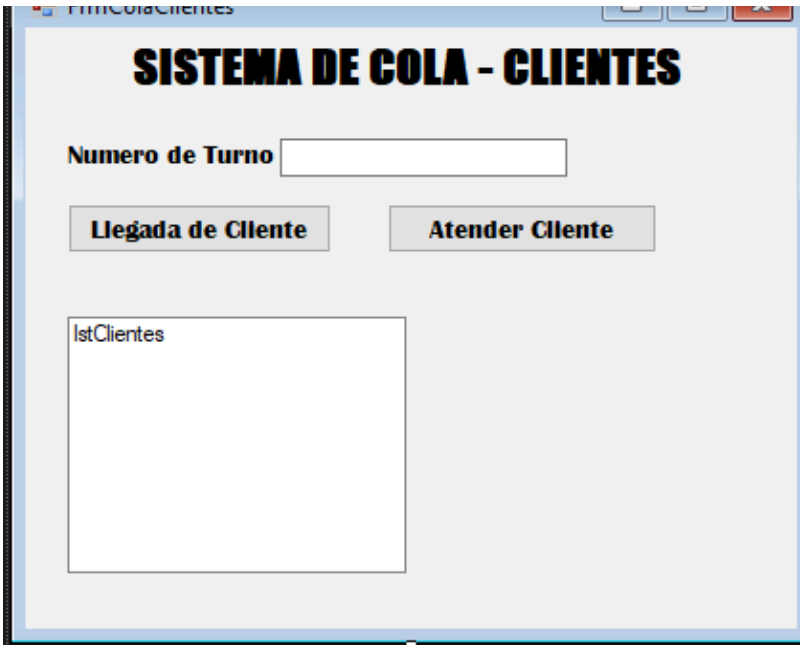
[5.1] Organización modular

Modulos	Clase	Formulario	Operacion
ListaSimple	NodoCliente, ListaSimpleClientes	FrmColaClientes	Op. 1: fila de clientes
ListaSimple	NodoMercancia, ListaSimpleMercancia	FrmMercaderia	Op. 2: mercadería recibida
ListaDoble	NodoDoblePago, ListaDoblePagos	FrmPagos	Op. 3: registrar y anular pagos; Op. 5: corregir historial
ListaCircular	NodoCategoria, ListaCircularRonda	FrmRonda	Op. 4: ronda de revisión
Proyecto principal	Program, FrmMenuPrincipal	FrmMenuPrincipal	Se visualisa y ejecuta las operaciones

[5.2] Evidencia modular



Formulario principal



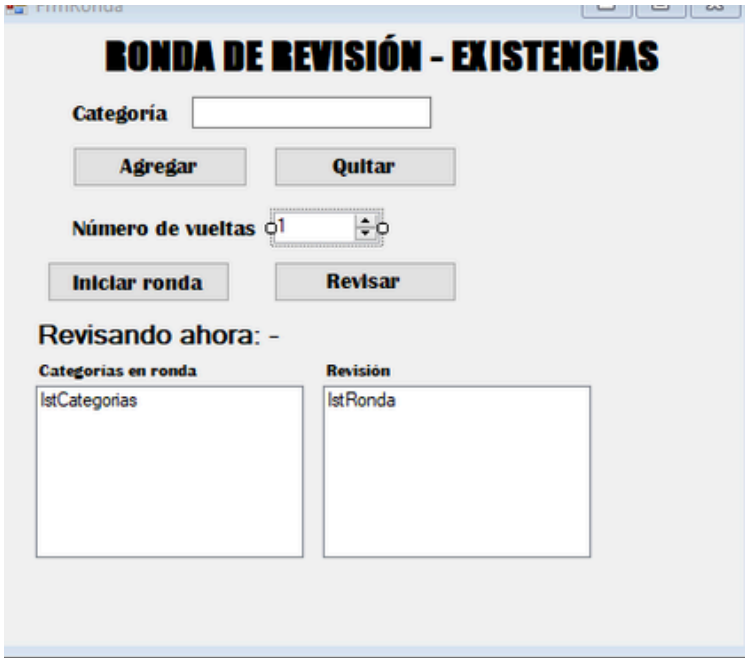
Formulario FrmClientes



Formulario FrmMercaderia



Formulario FrmPagos



Formulario FrmRonda

6. Casos de prueba :

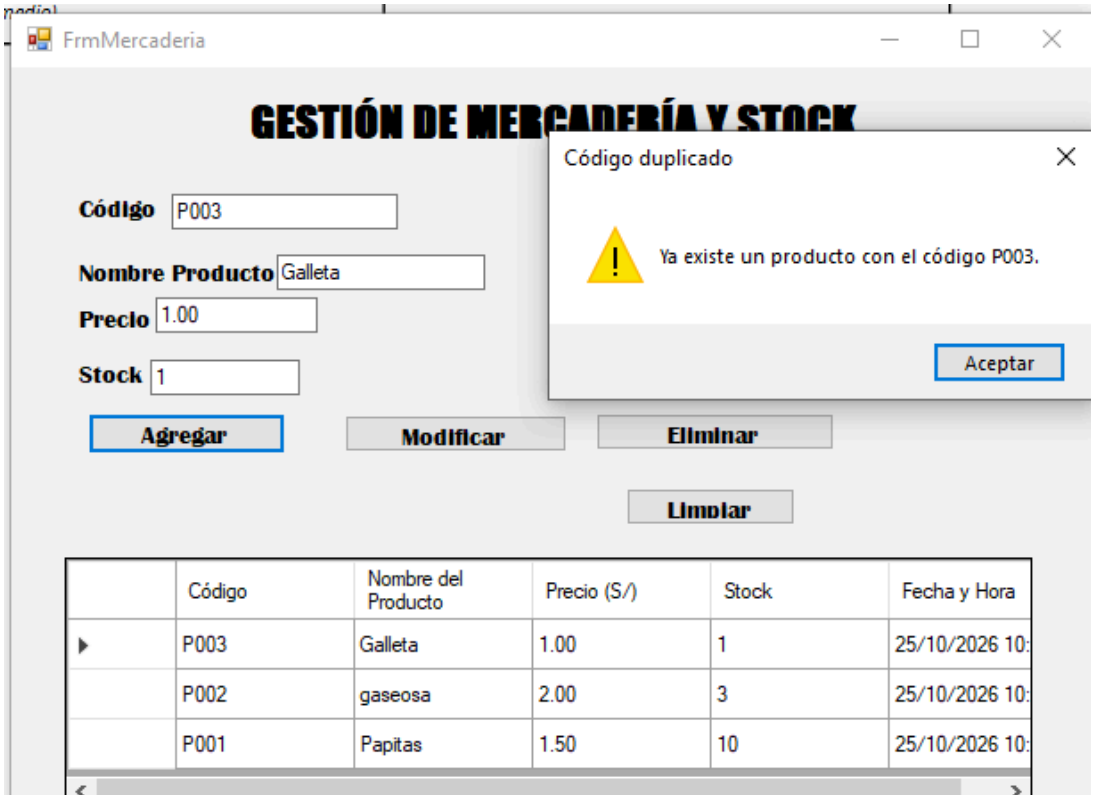
Se ejecuta el codigo y sus diferentes operaciones y consultas por código de producto, cada caso indica el método de la clase que se prueba, la entrada, el resultado esperado y un espacio para marcar el resultado y pegar la captura con fecha y hora visibles.

[6.1] Operación 1: Cola de clientes (FrmColaClientes)

método	Resultado esperado
C1-1 *InsertarDetras*	La lista muestra Turno 1, 2 y 3 en ese orden, cada uno con su hora de llegada.
C1-2 *EliminarPrimero, VerPrimero*	Mensaje «Atendiendo cliente – Turno: 1» con su hora de llegada. La lista queda con Turno 2 y 3.
C1-3 *InsertarDetras*	Los cuatro se agregan al final sin mover a los anteriores.
C1-4 *ExisteTurno*	Aviso «El turno número 9 ya se encuentra en la cola». La lista no cambia (6 clientes).
C1-5 *int.TryParse (validación)*	Aviso «Por favor, ingresa un número de turno válido». No se agrega ningún cliente.
C1-6 *EliminarPrimero, ListaVacia*	Se atiende en orden de llegada: 2, 3, 8, 9, 10, 11. A la 7.ª vez aparece «La cola de clientes está vacía» y la lista queda vacía.

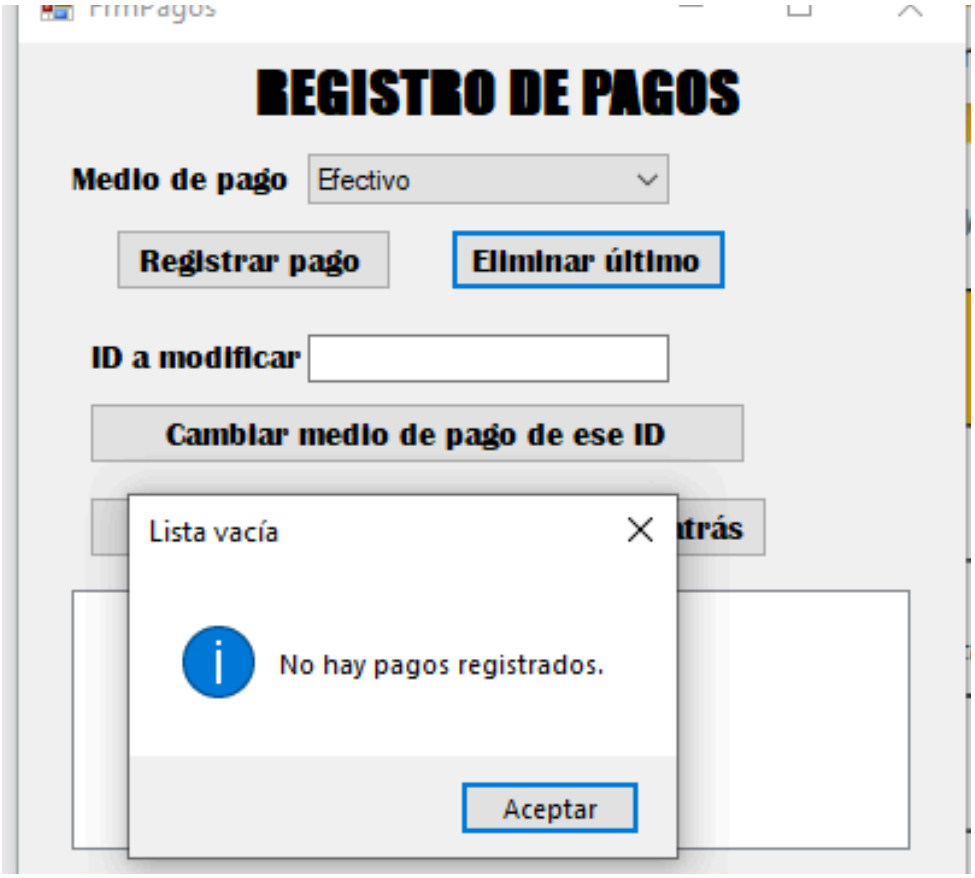
[6.2] Operación 2: Mercadería recibida (FrmMercaderia)

método	Resultado esperado
C2-1 *InsertarAlInicio*	La tabla muestra P003, P002, P001: el último ingreso queda primero
C2-2 *ExisteCodigo, Buscar*	Aviso «Ya existe un producto con el código...» en ambos intentos ,sigue con 3 productos.
C2-3 *Modificar*	Aviso «Producto modificado correctamente». P001 queda con stock ; nombre y precio no cambian y sigue en su posición.
C2-4 *Modificar, Buscar*	Aviso «Ya existe otro producto con el código P003». No se modifica nada.
C2-5 *Eliminar (nodo del medio)*	Aviso «Producto eliminado de la lista»



[6.3] Operación 3: Registrar y anular pagos (FrmPagos)

método	Resultado esperado
C3-1 *InsertarDetras*	IDs 1 a 11 crecientes, cada uno con su medio. Totales: 6 Yape, 1 Plin y 4 Efectivo, igual que la sección 2.
C3-2 *MostrarHaciaAdelante, MostrarHaciaAtras*	Hacia atrás: ID 11 → 1. Hacia adelante: ID 1 → 11. Son los mismos 11 registros.
C3-3 *EliminarUltimo*	Mensaje con ID 11 y medio Yape. La lista queda con los ID 1 a 10 y, hacia atrás, empieza en el ID 10.
C3-4 *InsertarDetras*	El nuevo pago recibe el ID 12 (el ID 11 no se reutiliza) y queda al final de la lista.
C3-5 *EliminarUltimo, ListaVacia*	Al quitar el último nodo que queda, la lista queda vacía sin error. A la vez siguiente: «No hay pagos registrados».



[6.4] Operación 4: Ronda de revisión (FrmRonda)

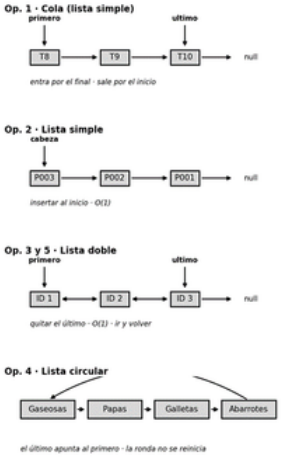
método	Resultado esperado
C4-1 *AgregarCategoria*	«Categorías en ronda» muestra las 4 en el orden de ingreso
C4-2 *ExisteCategoria*	Aviso «La categoría “gaseosas” ya está en la ronda» y aviso de campo vacío.
C4-3 *MostrarRondaRevision*	Revisión: Gaseosas, Papas, Galletas, Abarrotes (4 líneas).
C4-4 *MostrarRondaRevision*	8 líneas: Gaseosas, Papas, Galletas, Abarrotes, Gaseosas, Papas, Galletas, Abarrotes. Al terminar Abarrotes vuelve a Gaseosas sin reiniciar.
C4-5 *RevisarSiguiente*	Se agregan: Gaseosas, Papas, Galletas, Abarrotes y, en la 5.ª, de nuevo Gaseosas. La etiqueta termina en «Revisando ahora: Gaseosas».
C4-6 *EliminarCategoria (la que toca)*	Categorías: Gaseosas, Galletas, Abarrotes. La revisión pasa a Galletas
C4-7 *EliminarCategoria (última y no existente)*	Abarrotes sale y Galletas queda apuntando a Gaseosas. Para «Frutas»: aviso «No existe la categoría “Frutas” en la ronda». Tres «Revisar siguiente» dan: Gaseosas, Galletas, Gaseosas.
C4-8 *EliminarCategoria (hasta vaciar), ListaVacia*	Lista vacía, etiqueta «Revisando ahora: -» y aviso «Primero agrega al menos una categoría a la ronda».

[6.5] Operación 5: Revisar y corregir el historial (FrmPagos)

método	Resultado esperado
C5-1 *ModificarMedioPago*	Mensaje «El pago 3 ahora es: Yape.». Solo cambia el medio; el ID y la posición son los mismos. Totales: 7 Yape, 1 Plin y 3 Efectivo.
C5-2 *MostrarHaciaAdelante, MostrarHaciaAtras*	El ID 3 aparece con Yape en ambos sentidos y los 11 registros siguen enlazados, sin pérdidas.
C5-3 *ModificarMedioPago (extremos)*	Cambian solo esos dos pagos; el resto de la lista no se altera.
C5-4 *ModificarMedioPago (no existe)*	Aviso «No existe un pago con el ID 14». La lista no cambia.
C5-5 *int.TryParse (validación)*	Aviso «Por favor, ingresa un ID válido». No cambia nada.

7. Síntesis visual para la sustentación :

cada operación usa la estructura que coincide con lo observado: orden de llegada → cola; ingresos que solo crecen → lista simple; anular el último pago → lista doble; ronda repetida → lista circular.

PROBLEMA	ESTRUCTURAS	LÓGICA APLICADA
Bodega en el cruce del Jr. Manuel de Lara con Arias Aráguez, atendida por una sola persona. **11** pagos en 19 minutos (4 sep. 2026, 5:40–5:59 p. m.) **6·1·4** Yape · Plin · efectivo **3 y 4** personas en cola a las 5:40 y a las 5:59 Yape y Plin dependen del internet: si la señal falla o va lenta, el cliente y los que están detrás esperan y la venta puede quedar pendiente o mal anotada. **Propuesta: registrar la venta en el equipo asi se optimizaria la gestion de ventas y el manejo de los clientes de forma efectiva**	 *rente a un arreglo estático: la memoria crece y se libera nodo a nodo, sin desborde en hora pico, sin desperdicio en hora valle y sin desplazar elementos.*	**Op. 1 · Cola de clientes** • Encolar: ultimo.Siguiente = nuevo; ultimo = nuevo → O(1) • Atender: primero = primero.Siguiente → O(1) **Op. 2 · Mercadería** • Insertar al inicio: nuevo.Siguiente = cabeza; cabeza = nuevo → O(1) • Consulta por código: recorre la lista **Op. 3 · Pagos (lista doble)** • Anular el último: ultimo = ultimo.Anterior → O(1) **Op. 4 · Ronda (circular)** • El último apunta al primero; la ronda sigue sin reiniciar **Op. 5 · Corregir historial** • Buscar el ID recorre la lista; cambiar el dato es O(1) y los enlaces no se tocan

ANEXOS

reporsitorio git hub: https://github.com/Cris0cl12/G4_EstructurasDeDatos_EF.git

NodoCliente:

```
namespace Proyec.ListaSimple
{
  public class NodoCliente
  {
    //operación 1
    public int Turno;
    public DateTime Hora;
    public NodoCliente Siguiente;
    public NodoCliente(int turno, DateTime hora)
    {
      this.Turno = turno;
      this.Hora = hora;
      this.Siguiente = null;
    }
  }
}
```

ListaSimpleClientes:

```
namespace Proyec.ListaSimple
{
  public class ListaSimpleClientes
  {
    //operación 1
    private NodoCliente primero;
    private NodoCliente ultimo;
    public ListaSimpleClientes()
    {
      this.primero = null;
      this.ultimo = null;
    }
    public bool ListaVacia()
    {
      return this.primero == null;
    }
    public void InsertarDetras(int turno, DateTime hora)
    {
      NodoCliente nuevo = new NodoCliente(turno, hora);
      if (this.ListaVacia())
      {
        this.primero = nuevo;
        this.ultimo = nuevo;
      }
      else
      {
        this.ultimo.Siguiente = nuevo;
        this.ultimo = nuevo;
      }
    }
    public NodoCliente EliminarPrimero()
    {
      if (this.ListaVacia())
        return null;
      NodoCliente atendido = this.primero;
      this.primero = this.primero.Siguiente;
      if (this.primero == null)
      {
        this.ultimo = null;
      }
      atendido.Siguiente = null;
      return atendido;
    }
    public NodoCliente VerPrimero()
    {
      return this.primero;
    }
    public int TotalClientes()
    {
      NodoCliente aux;
      int total = 0;
      aux = this.primero;
      while (aux != null)
      {
        total += 1;
        aux = aux.Siguiente;
      }
      return total;
    }
    public void MostrarClientes(ListBox lst)
    {
      lst.Items.Clear();
      NodoCliente aux;
      aux = this.primero;
      while (aux != null)
      {
        //Aqui mostramos el turno y la hora de llegada del cliente en el ListBox
        lst.Items.Add($"Turno: {aux.Turno} - Hora: {aux.Hora.ToLongTimeString()}");
        aux = aux.Siguiente;
      }
    }
    public bool ExisteTurno(int turnoBuscado)
    {
      NodoCliente aux = this.primero;
      while (aux != null)
      {
        if (aux.Turno == turnoBuscado)
```

Problema transaccional en una bodega

NodoMercancia:

```
namespace Proyec.ListaSimple
{
    public class NodoMercancia
    {
        //operación 2
        public string Codigo;
        public string Nombre;
        public decimal Precio;
        public int Stock;
        public DateTime FechaHora;
        public NodoMercancia Siguiente;
        public NodoMercancia(string codigo, string nombre, decimal precio, int stock, DateTime fechaHora)
        {
            this.Codigo = codigo;
            this.Nombre = nombre;
            this.Precio = precio;
            this.Stock = stock;
            this.FechaHora = fechaHora;
            this.Siguiente = null;
        }
    }
}
```

ListaSimpleMercancia:

```
namespace Proyec.ListaSimple
{
    public class ListaSimpleMercancia
    {
        //operación 2
        private NodoMercancia cabeza;
        public ListaSimpleMercancia()
        {
            this.cabeza = null;
        }
        public bool ListaVacia()
        {
            return this.cabeza == null;
        }
        public void InsertarAlInicio(string codigo, string nombre, decimal precio, int stock, DateTime fechaHora)
        {
            NodoMercancia nuevo = new NodoMercancia(codigo, nombre, precio, stock, fechaHora);
            if (this.ListaVacia())
            {
                this.cabeza = nuevo;
            }
            else
            {
                nuevo.Siguiente = this.cabeza;
                this.cabeza = nuevo;
            }
        }
        private bool MismoCodigo(string a, string b)
        {
            return string.Equals(a, b, StringComparison.OrdinalIgnoreCase);
        }
        public NodoMercancia Buscar(string codigo)
        {
            NodoMercancia aux = this.cabeza;
            while (aux != null)
            {
                if (MismoCodigo(aux.Codigo, codigo))
                {
                    return aux;
                }
                aux = aux.Siguiente;
            }
            return null;
        }
        public bool ExisteCodigo(string codigo)
        {
            return Buscar(codigo) != null;
        }
        // Modifica el producto cuyo código es codigoActual.
        public bool Modificar(string codigoActual, string nuevoCodigo, string nombre, decimal precio, int stock)
        {
            NodoMercancia nodo = Buscar(codigoActual);
            if (nodo == null)
                return false;
            nodo.Codigo = nuevoCodigo;
            nodo.Nombre = nombre;
            nodo.Precio = precio;
            nodo.Stock = stock;
            return true;
        }
        public bool Eliminar(string codigo)
        {
            if (this.ListaVacia())
                return false;
            // Caso 1: el producto a eliminar es el primero
            if (MismoCodigo(this.cabeza.Codigo, codigo))
            {
                NodoMercancia primero = this.cabeza;
                this.cabeza = this.cabeza.Siguiente;
                primero.Siguiente = null;
                return true;
            }
            // Caso 2: está en medio o al final
            NodoMercancia anterior = this.cabeza;
            NodoMercancia aux = this.cabeza.Siguiente;
            while (aux != null)
            {
                if (MismoCodigo(aux.Codigo, codigo))
                {
                    anterior.Siguiente = aux.Siguiente;
                    aux.Siguiente = null;
                    return true;
                }
                anterior = aux;
                aux = aux.Siguiente;
            }
            return false;
        }
        public void MostrarMercancia(ListBox lst)
        {
            lst.Items.Clear();
 NodoMercancia aux = this.cabeza;
            while (aux != null)
            {
                lst.Items.Add($"Código: {aux.Codigo} - Producto: {aux.Nombre} - Precio: {aux.Precio} - Stock: {aux.Stock}");
                aux = aux.Siguiente;
            }
        }
        // Recorre la lista y la muestra en el DataGridView
        public void MostrarMercancia(DataGridView dgv)
        {
            DataTable tabla = new DataTable();
tabla.Columns.Add("Código", typeof(string));
            tabla.Columns.Add("Nombre del Producto", typeof(string));
            tabla.Columns.Add("Precio (S/)", typeof(decimal));
tabla.Columns.Add("Stock", typeof(int));
            tabla.Columns.Add("Fecha y Hora", typeof(DateTime));
            NodoMercancia aux = this.cabeza;
            while (aux != null)
            {
                tabla.Rows.Add(aux.Codigo, aux.Nombre, aux.Precio, aux.Stock, aux.FechaHora);
                aux = aux.Siguiente;
            }
            dgv.DataSource = tabla;
        }
    }
}
```

NodoDoblePago

```
namespace Proyec.ListaDoble
{
  public class NodoDoblePago
  {
    //operación 3
    public int IdTransaccion;
    public DateTime Hora;
    public string MedioPago;
    public NodoDoblePago Siguiente;
    public NodoDoblePago Anterior;
    public NodoDoblePago(int idTransaccion, DateTime hora, string medioPago)
    {
      this.IdTransaccion = idTransaccion;
      this.Hora = hora;
      this.MedioPago = medioPago;
      this.Siguiente = null;
      this.Anterior = null;
    }
  }
}
```

NodoCategoria

```
namespace Proyec.ListaCircular
{
  public class NodoCategoria
  {
    //operación 4
    public string Categoria;
    public NodoCategoria Siguiente;
    public NodoCategoria(string categoria)
    {
      this.Categoria = categoria;
      this.Siguiente = null;
    }
  }
}
```